ENGINEERING SOCIETY · TECHNICAL TEAM

Technical Blueprint

The full data model, the real technology stack, and why each decision was made. This is what we build against, locked.

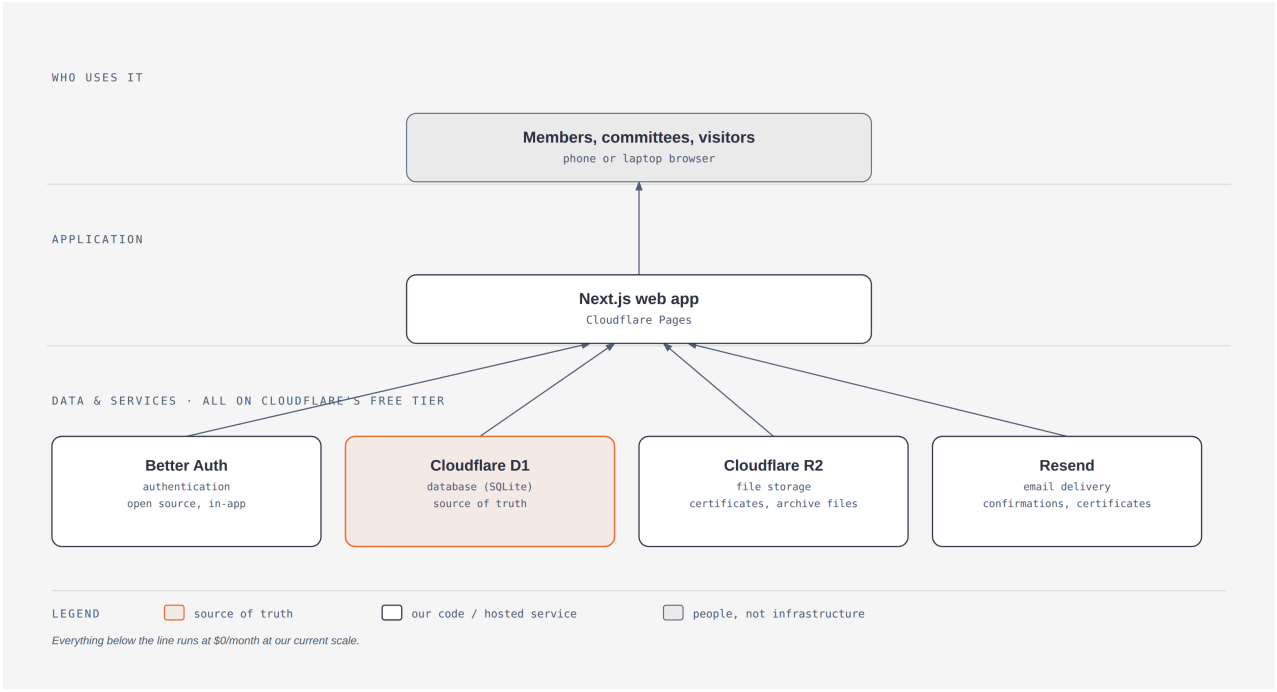
Version 1.0 · Prepared by Hisham Alhinai, Technical Team Lead

September 2026

PART 1 · THE STACK

The real technology, top to bottom

Five pieces, all free at our current scale, chosen to keep a small, mostly non-technical team productive with AI assistance, without needing to manage a server or pay for infrastructure we don't need yet.



Architecture decisions

Short and written down, so the reasoning survives even if the person who made the call isn't in the room.

ADR-01: Next.js on Cloudflare Pages

Decision: Build the member-facing site and forms in Next.js, deployed on Cloudflare Pages.

Why: Cloudflare Pages has no server for us to patch, back up, or lose sleep over, which matters with a team that's still learning. Its free tier comfortably covers a society-sized platform. Next.js remains the framework with the best AI coding support, so Salim, Ibrahim, and Zaher get the most help while learning.

ADR-02: Better Auth for authentication

Decision: Use Better Auth, an open-source authentication library, instead of a paid vendor auth service.

Why: it runs inside our own codebase rather than locking member accounts into a third-party platform, and it's free regardless of how many members we have. It integrates directly with our own database, so login and the Member record (UC-01) stay in one place, not split across two systems.

ADR-03: Cloudflare D1 for the database, R2 for files

Decision: Store all core data (Member, Event, Registration, Certificate, Archive item) in Cloudflare D1, and store files, certificates and archive uploads, in Cloudflare R2.

Why: both sit in the same platform as our hosting and are free at our scale, with R2 charging no fee to retrieve files, unlike most storage providers. D1 is SQLite rather than full Postgres, which is a real trade-off, but nothing in our seven use cases needs Postgres-only features, so we're not paying complexity for a capability we don't use.

ADR-04: Resend for email

Decision: Send confirmation and certificate emails through Resend rather than a mail server we manage.

Why: registration and certificate emails (UC-01, UC-04) need to arrive reliably. Resend's free tier covers our current volume, and running our own mail server is real infrastructure work with no learning value for the team.

Considered and set aside: our own VPS

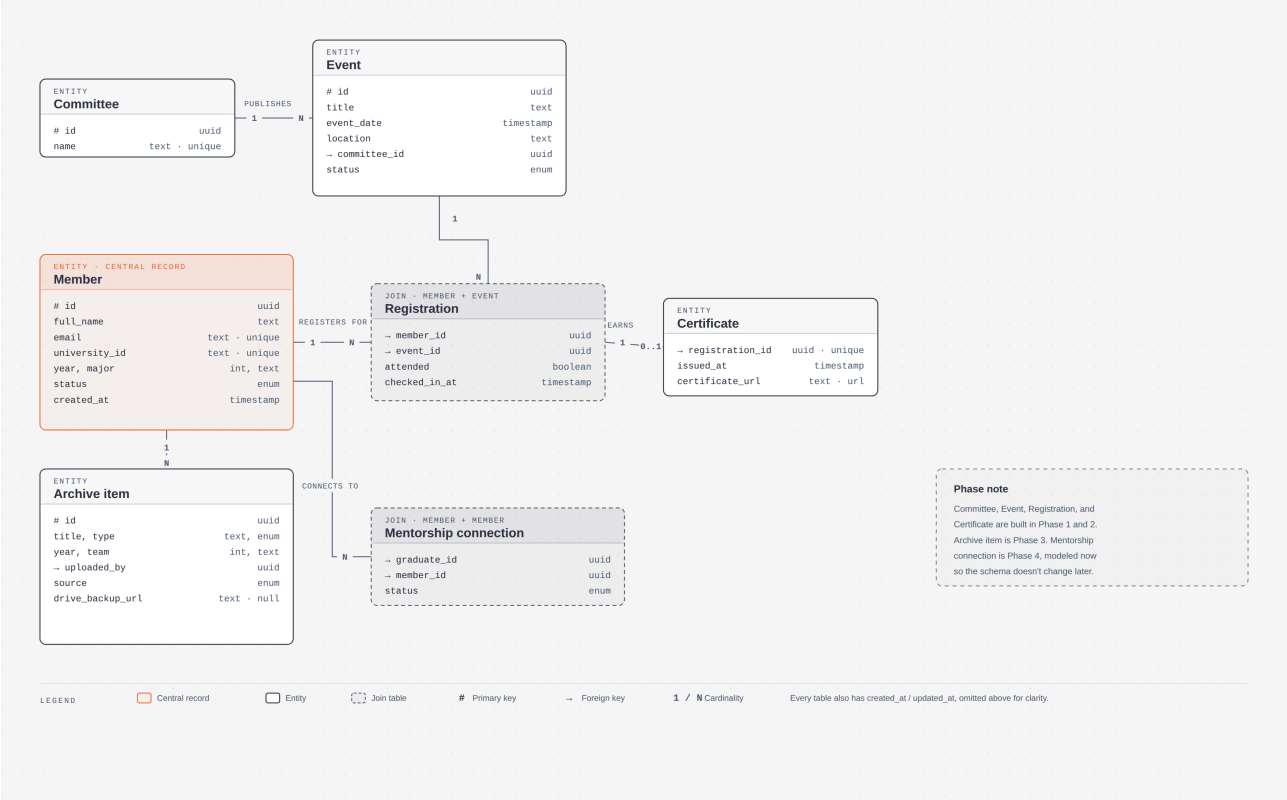
We looked at hosting everything ourselves on our VPS instead.

Why not, for now: a VPS gives full control, but it also makes us responsible for security patches, backups, and uptime, real sysadmin work with no safety net if something breaks at 2am before an event. Cloudflare removes that responsibility entirely for $0. If we outgrow the free tier or need something Cloudflare can't do, the VPS option stays open, this isn't a permanent door closing.

PART 2 · THE DATA MODEL

Full data model, v1.0

Every entity, every field, every connection, covering all seven approved use cases. This is the technical version of the simplified diagram we walked through as a team, same shape, full detail.



Seven entities. Committee, Event, Registration, and Certificate are built in Phase 1 and 2. Archive item is Phase 3. Mentorship connection is Phase 4, modeled now so the schema doesn't need to change later, but not built until then.

Field notes worth knowing

Every unique field is enforced by the database.

email and university_id on Member, name on Committee. This is what makes UC-01's duplicate detection reliable, the database itself refuses a second row with the same email, we don't have to catch it in application code.

Registration is the busiest table in the system.

Every event registration, every attendance check-in, and indirectly every certificate all pass through this one table. If something's wrong with attendance or certificates, this is the first place to look.

Certificate points to Registration, not to Member or Event directly.

That one link already knows which member and which event, since Registration links both. Storing member_id or event_id again on Certificate would just be the same fact written twice, and the two copies could quietly disagree with each other over time.

Archive item's drive_backup_url is nullable on purpose.

Items uploaded straight to the platform (UC-05) never had a Drive link. Only items migrated from Drive (UC-06) fill this field in. Nullable here means "not every row needs this," not "we forgot to fill it in."